

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 20

Total Marks:100

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I K. Manoj / 192422266 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%				
Explanation	25%				
Application	20%				
Organization	15%				
Accuracy	10%				
Clarity	5%				

Faculty Incharge

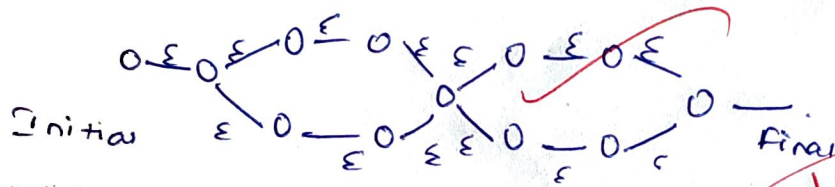
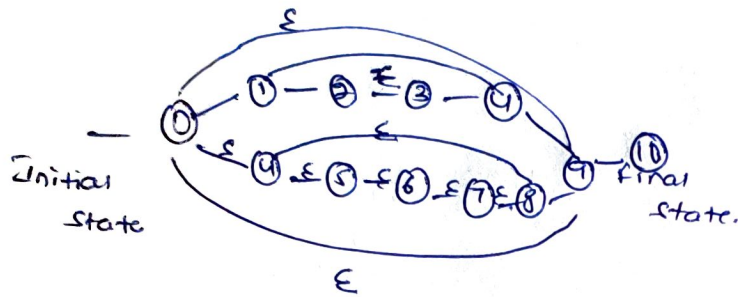
Name: K. Manoj Kumar Reddy.
Reg. No: 192425266
Date: 4/07/2026

Sub code: CSA1005

Sub Name: Computer Design.

COA-AT2

① $(a|b)^* a (a|b) | (a|b) \rightarrow$ Regular Expression.



② Predictive parsing.

$S \rightarrow (L) | a$

$L \rightarrow L, S | \epsilon$

$S \rightarrow (L) | a$

$L \rightarrow L, S | \epsilon$

$L \rightarrow L, S$

$L \rightarrow \epsilon$

$L \rightarrow SL' \rightarrow \textcircled{2}$

$L' \rightarrow SL' / \epsilon \rightarrow \textcircled{2}$

First

$\text{First}(S) = \{ (, a \}$

$\text{First}(L) = \{ (, a \}$

$\text{First}(L') = \{ (, a \}$

$\text{First}(\epsilon) = \{ \epsilon \}$

$\text{Follow}(S) = \{ (, a, \epsilon \}$

$\text{Follow}(L) = \{ (, a, \epsilon \}$

$\text{Follow}(L') = \{ (, a, \epsilon \}$

591

2

59/100

64/100

4

$$L \rightarrow SL1 \rightarrow ①$$

$$L \rightarrow SL1/\epsilon$$

② Predictive Parser.

$$S \rightarrow SS+ / SS* / a$$

string "aa+a"

$$S \rightarrow SS+$$

$$S \rightarrow SS*$$

$$S \rightarrow a$$

left recursion

$$S \rightarrow S+S'$$

$$S \rightarrow S+S'$$

$$S \rightarrow a$$

$$E \rightarrow c+c | c \times c | \epsilon$$

$$S' \rightarrow S a A'$$

$$S' \rightarrow S a A' / S b A' / \emptyset A'$$

$$S' \rightarrow S a A' | \epsilon$$

$$S \rightarrow S a / S b | \epsilon$$

It is possible to modify the grammar in any way to construct predictive form.

④ $S \rightarrow ca|cb|c$ — left recursion.

$$S' \rightarrow caa'_1$$

$$S' \rightarrow caa'_1 | cba'_1 | caa'_1 \text{ w.r.t. } a$$

$$S' \rightarrow caa'_1 / \epsilon \text{ — ①}$$

$$S' \rightarrow cba'_1 \text{ w.r.t. } b$$

$$S' \rightarrow cba'_1 / \epsilon \text{ — ②}$$

⑤ $A \rightarrow abcd|abce|f$ — Predictive Parsing.

$$A \rightarrow abcd$$

$$A \rightarrow abce$$

$$A \rightarrow f$$

$$A \rightarrow abcd \text{ — ①}$$

$$A \rightarrow abce \text{ — ②}$$

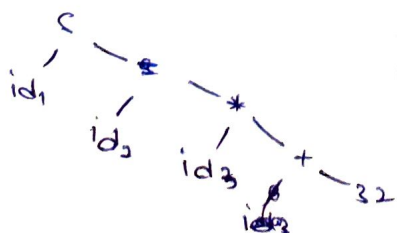
$$A \rightarrow f \text{ — ③}$$

⑥ Degree — $f = \text{Degree} - C * 1.8 + 32$ — Phases of a compiler.

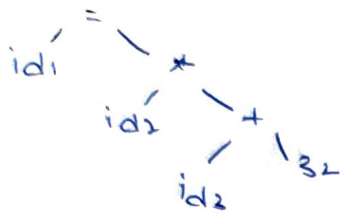
(i) Lexical Analyzer:

$\{id_1\} \{id_2\} \{id_3\} \{id_4\} \{id_5\}$

(ii) Syntax Analyzer =



(ii) Symmetric Analyzer:



(iv) Intermediate:

temp1 = int to real
temp2 = id1 * temp1
temp3 = id2 + temp2
id1 = temp3

(v) Code optimizer:

MOVF id1, r2
MOVF #1.8
MOVF id2 * 1.8 + 32
MOVF id2, r2
MOVF r0, r1

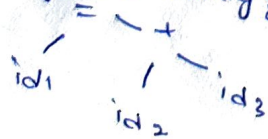
⑦ sum = sum + i

phases of compiler.

$\{id1\} \Rightarrow \{id2\} + \{id3\}$

(i) Lexical Analyzer.

(ii) Syntax Analyzer



(iii) Intermediate

temp1 = id2 + id3
temp2 = id1
temp3 = id1, id3
id1 = temp3

(iv) Code generator.

MOV r, id2

MOV r, id2 + id2

MOV r, id3 + r_L

MOV r, id1

MOV r_L, r_D(8) $S \rightarrow aABb, A \rightarrow c/\epsilon, B \rightarrow d/\epsilon$ $S \rightarrow aABb$ $A \rightarrow c/\epsilon$ $B \rightarrow d/\epsilon$ $S \rightarrow aABb$ $A \rightarrow c/\epsilon$ $B \rightarrow d/\epsilon$ $\text{First}(S) = \{c/\epsilon\}$ $\text{First}(A) = \{c/\epsilon\}$ $\text{First}(B) = \{d, \epsilon\}$ $\text{Follow}(S) = \{ \}$ $\text{Follow}(A) = \{d, b\}$ $\text{Follow}(B) = \{d, b\}$

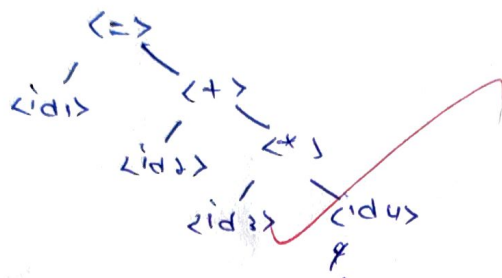
Predictive parsing table

	c	d	d	ϵ	ϵ
S	$S \rightarrow aABb$				
A	$A \rightarrow c$		$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$
B		$B \rightarrow d$	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$

⑨ Lex program.

sample input: $a = b + c * d$

$\langle id_1 \rangle \langle = \rangle \langle id_2 \rangle \langle + \rangle \langle id_3 \rangle \langle * \rangle \langle id_4 \rangle$



Code:

$MOV(r_1) = id_1, id_2, id_3, id_4$

ADD $r_1 =$

$MOV(r_2, r_1) = id_2 * id_3$

$ADD(r_1, r_2) = r_2 + id_4$

$RES(r_1, r_2) = id_1$

⑩ Eliminate the left recursion

$A \rightarrow A\alpha / \beta$

$A \rightarrow A\alpha$

$A \rightarrow \beta$

where β does not belong to A

Eliminate left recursion by replacing their pair of products with

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' / \epsilon$

Elimination of left factoring.

if,

$A \rightarrow \alpha A'$

$A' \rightarrow \beta / \gamma$

where α does not belong to A

Eliminate left factoring by replacing their pair.

$A \rightarrow \alpha A'$

① shift Reduce parsing.

→ type of bottom up parsing.

Operations

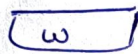
① shift → moving current symbol from i/p (input) onto the stack.

② Reduce → POP out RHS of the production rule from stack & push LHS of production to the stack.

③ Accept → includes \$ symbol, the i/p string is accepted.

④ Error → If shift & reduce not possible display error.

$S \rightarrow a + c$



i/p buffer.

3
2

Rule 1: If priority of the operator is higher than operator at the stack perform shift.

Rule 2: If operator is equal or less than, perform reduced.
(id, (, *, #, €, \$)

12) $S \rightarrow PR$

$R \rightarrow qB/r$

?

$$13. E \rightarrow E + T / T$$

$$T \rightarrow T * F / F$$

$$F \rightarrow (E) / id$$

$$1) E \rightarrow E + T / T$$

$$E \rightarrow E T$$

$$E \rightarrow E T F$$

$$2) T \rightarrow T * F / F$$

$$T \rightarrow T F$$

$$T \rightarrow T F F$$

$$3) F \rightarrow (E) + id$$

$$F \rightarrow id +$$

$$F \rightarrow (E) + id$$

$$\text{first}(E) = \{ (, + \}$$

$$\text{first}(E') = \{ +, , \}$$

$$\text{first}(T) = \{ (, id \}$$

$$\text{first}(T') = \{ +, , \}$$

$$\text{first}(F) = \{ (, id \}$$

$$\text{follow}(E) = \{ \$ \}$$

$$\text{follow}(E') = \{ \$, , \}$$

$$\text{follow}(T) = \{ +, \$ \}$$

$$\text{follow}(T') = \{ +, \$ \}$$

$$\text{follow}(F) = \{ +, \$ \}$$

14

$C \rightarrow AR$

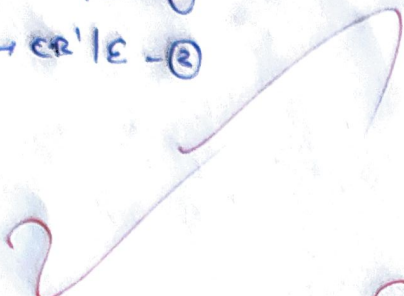
$A \rightarrow aA|a$

$B \rightarrow cR|c$

$C \rightarrow AR - ①$

$A' \rightarrow aA'| \epsilon - ②$

$B' \rightarrow cB'| \epsilon - ③$



15

15 $A \rightarrow aAB|aA|a$

$B \rightarrow bB|b$

$A \rightarrow aAB|aA|a$

$A \rightarrow aA'$

$A' \rightarrow aA'| \epsilon$

$A \rightarrow aAB$

$A \rightarrow aB$

$A' \rightarrow aAB|aA| \epsilon - ①$

$B \rightarrow bB|b - ②$

$B \rightarrow bB|b$

$B' \rightarrow bB'$

$B' \rightarrow bB'| \epsilon$



$A \rightarrow A'AB$

$A' \rightarrow aA'| \epsilon$

$A \rightarrow aA$

$A \rightarrow aA'$

$A' \rightarrow aA'| \epsilon$

19) $S \rightarrow ABC$

$A \rightarrow a/\epsilon$

$B \rightarrow r/\epsilon$

$C \rightarrow b/\epsilon$

$S \rightarrow ABC - ①$

$A \rightarrow a/\epsilon - ②$

$B \rightarrow r/\epsilon - ③$

$C \rightarrow b/\epsilon - ④$

$\text{first}(S) = \{ABC\}$

$\text{first}(A) = \{a, \epsilon\}$

$\text{first}(B) = \{r, \epsilon\}$

$\text{first}(C) = \{b, \epsilon\}$

$\text{follow}(S) = \{\$ ABC\}$

$\text{follow}(A) = \{\$, a, c\}$

$\text{follow}(B) = \{\$, r, \epsilon\}$

$\text{follow}(C) = \{\$, b, \epsilon\}$

20) $S \rightarrow 1S4/qABC$

$A \rightarrow a/bbD$

$Q \rightarrow a/\epsilon$

$C \rightarrow b/\epsilon$

$D \rightarrow r/\epsilon$

$S \rightarrow 1S4/qABC - ①$

$S \rightarrow qABC - ②$

$A \rightarrow a - ③$

$A \rightarrow bbD - ④$

$A \rightarrow a - ⑤$

$C \rightarrow b - ⑥$

$D \rightarrow r - ⑦$